Diet Tracking System

Final Practical Assignment — Database

Group: Bellili Fouad · Blaszczyk Jakub **Course:** Bases de datos **Academic Year:** 2025/2026
Submission Date: June 3, 2026

---

1. Requirements Analysis

1.1 Context

The **Diet Tracking System** is a nutritional tracking application that allows users to log their daily food intake, follow personalized diet plans created by nutritionists, and monitor their weight evolution over time. The system also supports subscription plans with different feature tiers and records associated payments.

1.2 Functional Requirements

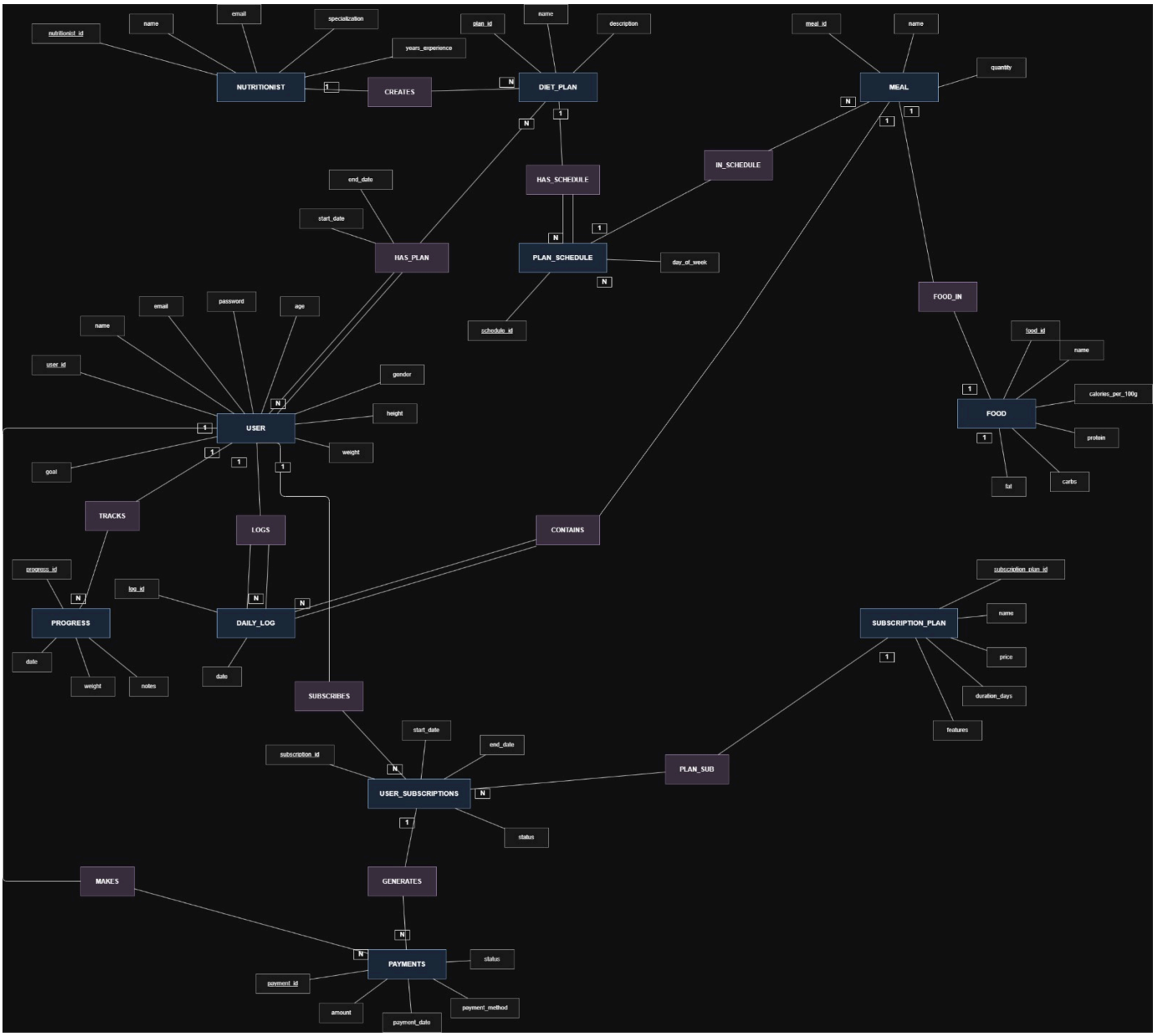
ID	Requirement
RF01	The system must allow user registration and authentication
RF02	Users must be able to browse and subscribe to diet plans
RF03	Users must be able to log meals in a daily food diary
RF04	The system must automatically calculate consumed calories and macronutrients
RF05	Users must be able to record their weight and track progress over time
RF06	Nutritionists must be able to create and manage weekly diet plans
RF07	The system must support subscription plans with different feature sets
RF08	Payments must be recorded and linked to subscriptions
RF09	Subscription statuses must be updated automatically
RF10	The system must calculate the user's BMI based on current profile data

1.3 Non-Functional Requirements

- The database must run on SQL Server (MSSQL)
- User passwords must be stored in hashed form
- Only one food diary is allowed per user per day
- Nutritional values are stored per 100g and extrapolated by meal quantity

---

2. Entity-Relationship Diagram (ERD)



Main entities:

- `nutritionists` — professionals who create diet plans
- `foods` — food items with nutritional values per 100g
- `subscription_plans` — available subscription tiers
- `diet_plans` — diet plans created by nutritionists
- `meals` — a food item combined with a quantity
- `plan_schedules` — weekly scheduling of a diet plan
- `users` — registered users of the platform
- `progress` — weight records over time
- `daily_logs` — daily food diary of a user
- `user_subscriptions` — user subscriptions
- `payments` — payments linked to subscriptions

Association tables (junction tables):

- `plan_schedule_meals` — meals assigned to a day within a plan
- `user_diet_plans` — diet plans followed by a user
- `log_meals` — meals recorded in a daily log

---

3. Relational Schema

```
nutritionists (nutritionist_id PK, nutritionist_name, email UQ, specialization, years_of_experience)

foods (food_id PK, food_name UQ, calories_per_100g, protein_per_100g, carbs_per_100g, fat_per_100g)

subscription_plans (subscription_plan_id PK, subscription_name UQ, price, duration_days, features)

diet_plans (diet_plan_id PK, plan_name UQ, description, nutritionist_id FK→nutritionists)

meals (meal_id PK, meal_name UQ, quantity, food_id FK→foods)

plan_schedules (schedule_id PK, day_of_week, diet_plan_id FK→diet_plans)

plan_schedule_meals (schedule_id FK→plan_schedules, meal_id FK→meals)  PK(schedule_id, meal_id)

users (user_id PK, user_name UQ, email UQ, password, age, gender, height, weight, goal)

user_diet_plans (user_id FK→users, diet_plan_id FK→diet_plans, start_date, end_date)
  PK(user_id, diet_plan_id)

progress (progress_id PK, date, weight, notes, user_id FK→users)  UQ(user_id, date)

daily_logs (log_id PK, date, user_id FK→users)  UQ(user_id, date)

log_meals (log_id FK→daily_logs, meal_id FK→meals)  PK(log_id, meal_id)

user_subscriptions (subscription_id PK, start_date, end_date, status,
  user_id FK→users, subscription_plan_id FK→subscription_plans)

payments (payment_id PK, amount, payment_date, payment_method, status,
  subscription_id FK→user_subscriptions)
```

---

4. Normalization

4.1 First Normal Form (1NF)

All tables satisfy 1NF:

- Every column holds atomic values — no repeating groups or arrays
- Every row is uniquely identified by a primary key
- The `features` column in `subscription_plans` is a descriptive string, not a structured list — acceptable in this context

4.2 Second Normal Form (2NF)

Tables with composite primary keys (`plan_schedule_meals`, `user_diet_plans`, `log_meals`) satisfy 2NF:

- In `user_diet_plans`, `start_date` and `end_date` depend on the full composite key `(user_id, diet_plan_id)` — no partial dependency
- `log_meals` and `plan_schedule_meals` are pure junction tables with no non-key attributes

4.3 Third Normal Form (3NF)

All tables satisfy 3NF:

- No transitive dependencies exist between non-key attributes
- Nutritional data per 100g is centralized in `foods` and not duplicated in `meals` — eliminates redundancy
- The user's current weight in `users` is synchronized automatically via trigger from `progress`

---

5. SQL DDL

The structure creation scripts are located in:

- `diet_tracking_system_tables.sql` — CREATE TABLE statements with primary and foreign keys
- `food_tracking_constrainst.sql` — ALTER TABLE statements with CHECK and UNIQUE constraints
- `diet_tracking_system_indexes.sql` — CREATE INDEX statements for query optimization

Example — central table:

```
CREATE TABLE users (
    user_id   INT PRIMARY KEY,
    user_name VARCHAR(255) NOT NULL,
    email     VARCHAR(255) NOT NULL UNIQUE,
    password  VARCHAR(255) NOT NULL,
    age       INT NOT NULL,
    gender    VARCHAR(20) NOT NULL,
    height    FLOAT NOT NULL,
    weight    FLOAT NOT NULL,
    goal      VARCHAR(255) NOT NULL
);
```

---

6. SQL DML — Queries Used in the Application

6.1 Daily summary for a user

Calls `sp_get_user_daily_summary`, which aggregates calories, protein, carbohydrates, and fat from all meals logged on a given day.

6.2 Diet plan assignment

Calls `sp_assign_diet_plan` with date overlap validation — prevents assigning the same plan twice over an overlapping period.

6.3 Payment registration and subscription activation

`sp_register_payment` inserts the payment record; the trigger `trg_payments_set_subscription_status` then automatically activates the linked subscription if the payment status is `completed`.

---

7. Indexes

Index	Table	Columns	Type	Justification
`ix_diet_plans_nutritionist_id`	`diet_plans`	`nutritionist_id`	Regular	Filter plans by nutritionist
`ix_meals_food_id`	`meals`	`food_id`	Regular	Frequent JOIN with `foods`
`ix_plan_schedules_diet_plan_id`	`plan_schedules`	`diet_plan_id`	Regular	Weekly schedule lookup
`ix_user_diet_plans_diet_plan_id`	`user_diet_plans`	`diet_plan_id`	Regular	Users following a plan

Index	Table	Columns	Type	Justification
ux_progress_user_id_date	progress	user_id, date	Unique	One weight record per user/day
ux_daily_logs_user_id_date	daily_logs	user_id, date	Unique	One log per user/day
ix_log_meals_meal_id	log_meals	meal_id	Regular	Reverse JOIN meal→log
ix_user_subscriptions_user_id_status_dates	user_subscriptions	user_id, status, start_date, end_date	Regular	Active subscription check
ix_user_subscriptions_plan_id	user_subscriptions	subscription_plan_id	Regular	Users per subscription plan
ix_payments_subscription_id_status	payments	subscription_id, status	Regular	Payment verification

8. Triggers

trg_payments_set_subscription_status

Table: payments — AFTER INSERT, UPDATE
When a payment with status completed is inserted or updated, the trigger automatically sets the linked subscription to active if it was pending or expired and the payment date falls within the subscription period.

trg_progress_update_user_weight

Table: progress — AFTER INSERT
When a new progress record is inserted, the trigger updates the user's current weight in the users table if the new entry is the most recent one — keeping the profile data in sync without manual updates.

trg_daily_logs_one_per_user_date

Table: daily_logs — AFTER INSERT, UPDATE
Prevents more than one daily log per user and date, complementing the unique index with an explicit error message.

9. Stored Procedures

Procedure	Description
sp_create_daily_log	Creates a new daily log with duplicate validation
sp_add_meal_to_log	Adds a meal to an existing daily log
sp_record_progress	Records the user's weight on a given date
sp_assign_diet_plan	Assigns a diet plan to a user with date overlap validation
sp_register_payment	Registers a payment for a subscription
sp_get_user_daily_summary	Returns the caloric and nutritional summary for a user on a given day
sp_update_subscription_statuses	Bulk-updates subscription statuses (expired / activated)

10. User-Defined Functions (UDFs)

Function	Return Type	Description
`fn_calculate_meal_calories(@meal_id)`	`DECIMAL(10,2)`	Calories of a meal based on quantity and food nutritional data
`fn_calculate_daily_log_calories(@log_id)`	`DECIMAL(10,2)`	Total calories of all meals in a daily log
`fn_calculate_user_bmi(@user_id)`	`DECIMAL(10,2)`	User BMI: weight / (height / 100)^2
`fn_user_has_active_subscription(@user_id, @date)`	`BIT`	Returns 1 if the user has an active subscription on the given date

11. Conclusion

This project resulted in a fully functional diet tracking system, built around a normalized relational database running on SQL Server and a C# Windows Forms application as the graphical interface.

The database design went through a complete development cycle: starting from requirements analysis and entity-relationship modelling, moving into a normalized relational schema (1NF, 2NF, 3NF), and then translated into SQL Server using dedicated DDL scripts for tables, constraints, indexes, views, stored procedures, UDFs, and triggers — each organized in its own file for clarity and reusability.

The use of stored procedures as the main data access layer keeps the business logic centralized in the database and decoupled from the application code. Triggers automate key consistency rules — such as activating subscriptions upon payment confirmation and keeping the user's weight in sync with their latest progress record — reducing the risk of data inconsistencies caused by application-level errors. User-defined functions encapsulate reusable calculations (BMI, meal calories, daily totals) that are called both from stored procedures and directly within the application. Indexes were chosen based on the most frequent query patterns (JOIN columns, filter columns, and uniqueness constraints) to improve performance without unnecessary overhead.

On the application side, all user interactions — logging meals, recording progress, subscribing to a plan, registering payments — are handled through the stored procedures, so the application never executes raw ad-hoc SQL. Connection string configuration is centralized in a single location in the project, documented in the README, making it straightforward to adapt for testing.

Overall, the project demonstrated how a well-structured relational database, combined with SQL Server's programmability features, can support a real-world application with complex rules around data integrity, automation, and multi-user access.

---

*Report written by Fouad Bellili and Blaszczyk Jakub — Universidade de Aveiro, 2025/2026